Documento de Engenharia de Software

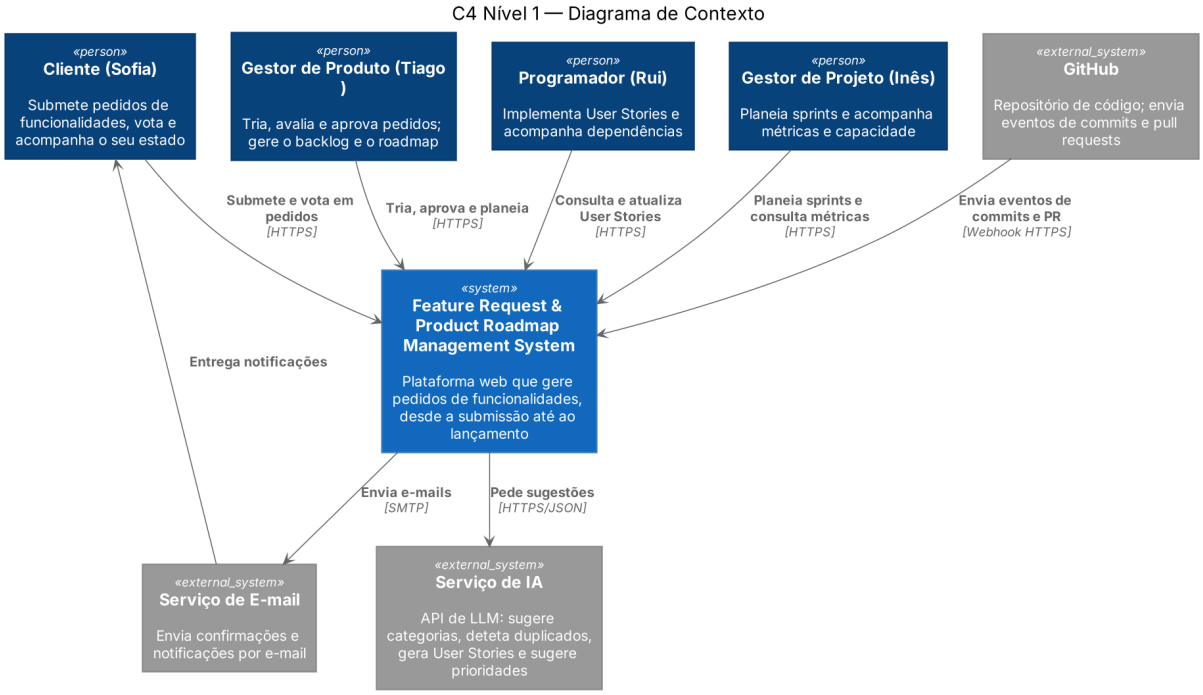
Feature Request & Product Roadmap Management System

Diagramas de Arquitetura (C4)

Ano Letivo 2026/27

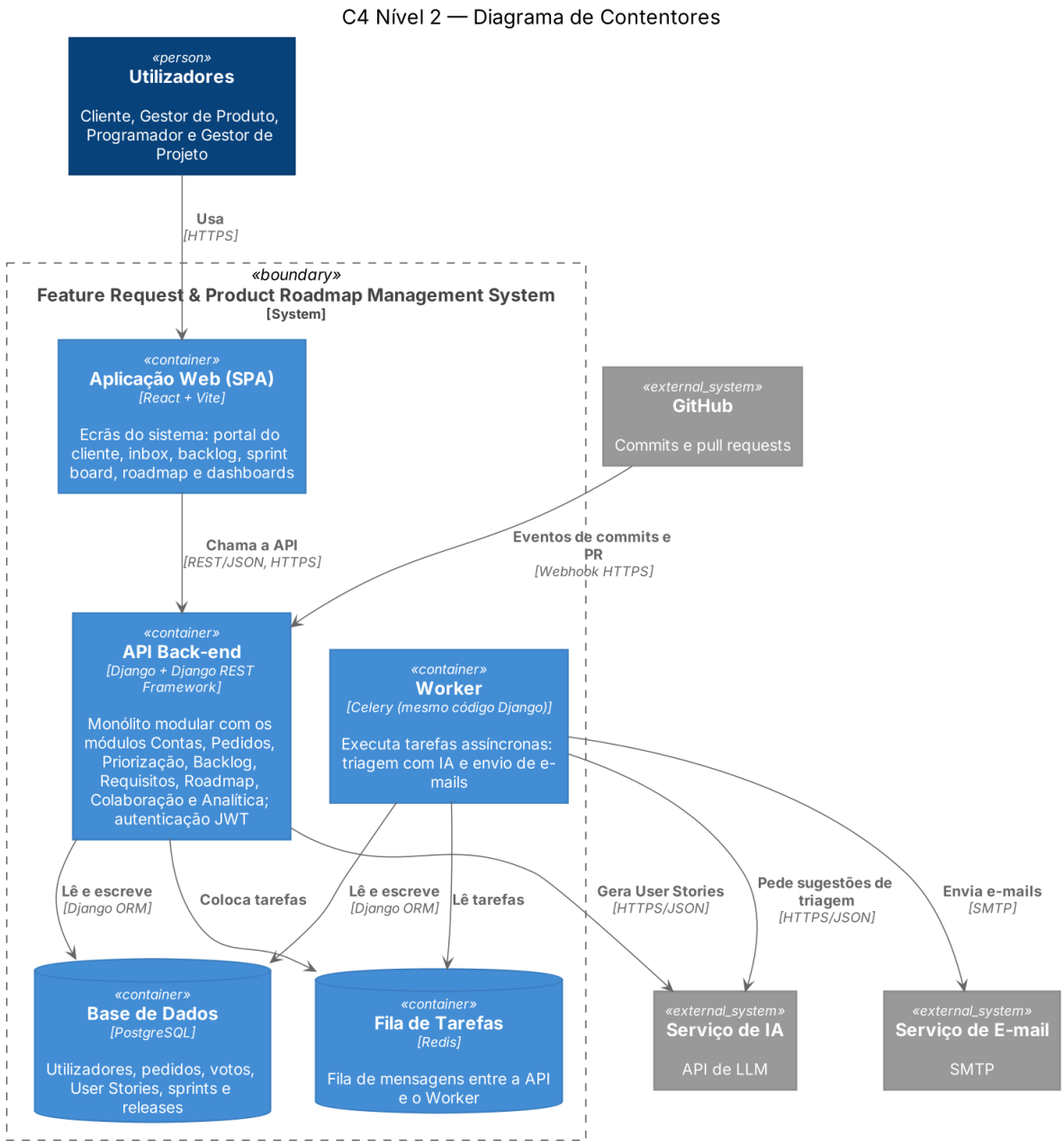
Nível 1 — Contexto

Mostra o sistema como uma única caixa, quem o usa (as quatro personas) e os sistemas externos com que comunica: o Serviço de IA, o GitHub e o Serviço de E-mail.



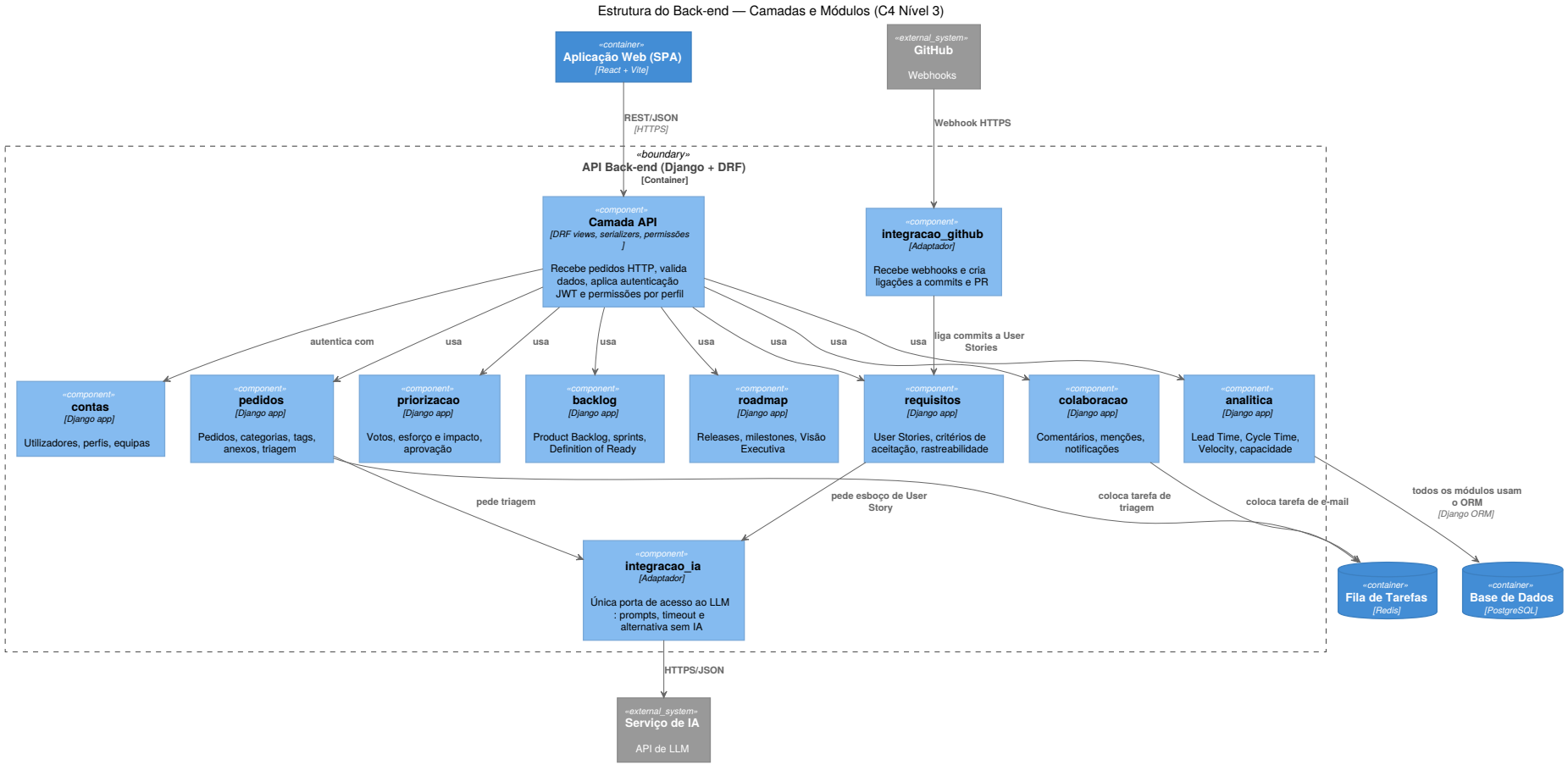
Nível 2 — Contentores

Abre o sistema e mostra as peças que correm separadamente: a aplicação web em React, a API em Django REST Framework (monólito modular), o Worker (Celery) com a fila de tarefas Redis para o trabalho assíncrono, e a base de dados PostgreSQL, com a tecnologia e o protocolo de cada ligação.



Nível 3 — Componentes

Abre o contentor API Back-end e mostra os componentes que o compõem: a Camada API, que recebe os pedidos HTTP, valida os dados e aplica a autenticação JWT e as permissões por perfil; as apps Django de cada área do domínio (contas, pedidos, priorizacao, backlog, requisitos, roadmap, colaboracao e analitica); e os dois adaptadores, integracao_ia e integracao_github, que são a única porta de saída para o Serviço de IA e para o GitHub. Todos os módulos usam a base de dados através do ORM do Django e colocam o trabalho assíncrono na fila de tarefas Redis.



Código PlantUML

c4_nivel1_contexto.puml

```
@startuml c4_nivel1_contexto
!include <C4/C4_Context>
skinparam dpi 150
skinparam defaultFontName Helvetica
title C4 Nível 1 -- Diagrama de Contexto

Person(cliente, "Cliente (Sofia)", "Submete pedidos de funcionalidades, vota e acompanha o seu estado")
Person(gprod, "Gestor de Produto (Tiago)", "Tria, avalia e aprova pedidos; gere o backlog e o roadmap")
Person(prog, "Programador (Rui)", "Implementa User Stories e acompanha dependências")
Person(gproj, "Gestor de Projeto (Inês)", "Planeia sprints e acompanha métricas e capacidade")

System(sistema, "Feature Request & Product Roadmap Management System", "Plataforma web que gere pedidos de funcionalidades, desde a submissão até ao lançamento")

System_Ext(ia, "Serviço de IA", "API de LLM: sugere categorias, deteta duplicados, gera User Stories e sugere prioridades")
System_Ext(github, "GitHub", "Repositório de código; envia eventos de commits e pull requests")
System_Ext(email, "Serviço de E-mail", "Envia confirmações e notificações por e-mail")

Rel(cliente, sistema, "Submete e vota em pedidos", "HTTPS")
Rel(gprod, sistema, "Tria, aprova e planeia", "HTTPS")
Rel(prog, sistema, "Consulta e atualiza User Stories", "HTTPS")
Rel(gproj, sistema, "Planeia sprints e consulta métricas", "HTTPS")

Rel(sistema, ia, "Pede sugestões", "HTTPS/JSON")
Rel(github, sistema, "Envia eventos de commits e PR", "Webhook HTTPS")
Rel(sistema, email, "Envia e-mails", "SMTP")
Rel(email, cliente, "Entrega notificações")
@enduml
```

c4_nivel2_contentores.puml

```
@startuml c4_nivel2_contentores
!include <C4/C4_Container>
skinparam dpi 150
skinparam defaultFontName Helvetica
title C4 Nível 2 -- Diagrama de Contentores

Person(utilizadores, "Utilizadores", "Cliente, Gestor de Produto, Programador e Gestor de Projeto")

System_Boundary(sistema, "Feature Request & Product Roadmap Management System") {
  Container(spa, "Aplicação Web (SPA)", "React + Vite", "Ecrãs do sistema: portal do cliente, inbox, backlog, sprint board, roadmap e dashboards")
  Container(api, "API Back-end", "Django + Django REST Framework", "Monólito modular com os módulos Contas, Pedidos, Priorização, Backlog, Requisitos, Roadmap, Colaboração e Analítica; autenticação JWT")
  Container(worker, "Worker", "Celery (mesmo código Django)", "Executa tarefas assíncronas: triagem com IA e envio de e-mails")
  ContainerDb(redis, "Fila de Tarefas", "Redis", "Fila de mensagens entre a API e o Worker")
  ContainerDb(db, "Base de Dados", "PostgreSQL", "Utilizadores, pedidos, votos, User Stories, sprints e releases")
}

System_Ext(ia, "Serviço de IA", "API de LLM")
```

```
System_Ext(github, "GitHub", "Commits e pull requests")
System_Ext(email, "Serviço de E-mail", "SMTP")

Rel(utilizadores, spa, "Usa", "HTTPS")
Rel(spa, api, "Chama a API", "REST/JSON, HTTPS")
Rel(api, db, "Lê e escreve", "Django ORM")
Rel(api, redis, "Coloca tarefas")
Rel(worker, redis, "Lê tarefas")
Rel(worker, db, "Lê e escreve", "Django ORM")
Rel(worker, ia, "Pede sugestões de triagem", "HTTPS/JSON")
Rel(api, ia, "Gera User Stories", "HTTPS/JSON")
Rel(worker, email, "Envia e-mails", "SMTP")
Rel(github, api, "Eventos de commits e PR", "Webhook HTTPS")
@enduml
```

Código PlantUML (nível 3)

componentes_backend.puml

```
@startuml componentes_backend
!include <C4/C4_Component>
skinparam dpi 96
skinparam defaultFontName Helvetica
title Estrutura do Back-end -- Camadas e Módulos (C4 Nível 3)

Container(spa, "Aplicação Web (SPA)", "React + Vite")
ContainerDb(db, "Base de Dados", "PostgreSQL")
ContainerDb(redis, "Fila de Tarefas", "Redis")
System_Ext(ia, "Serviço de IA", "API de LLM")
System_Ext(github, "GitHub", "Webhooks")

Container_Boundary(api, "API Back-end (Django + DRF)") {
  Component(rest, "Camada API", "DRF views, serializers, permissões", "Recebe pedidos HTTP,
    valida dados, aplica autenticação JWT e permissões por perfil")
  Component(contas, "contas", "Django app", "Utilizadores, perfis, equipas")
  Component(pedidos, "pedidos", "Django app", "Pedidos, categorias, tags, anexos, triagem")
  Component(prior, "priorizacao", "Django app", "Votos, esforço e impacto, aprovação")
  Component(backlog, "backlog", "Django app", "Product Backlog, sprints, Definition of Ready")
  Component(req, "requisitos", "Django app", "User Stories, critérios de aceitação,
    rastreabilidade")
  Component(roadmap, "roadmap", "Django app", "Releases, milestones, Visão Executiva")
  Component(colab, "colaboracao", "Django app", "Comentários, menções, notificações")
  Component(anal, "analitica", "Django app", "Lead Time, Cycle Time, Velocity, capacidade")
  Component(iaad, "integracao_ia", "Adaptador", "Única porta de acesso ao LLM: prompts,
    timeout e alternativa sem IA")
  Component(ghad, "integracao_github", "Adaptador", "Recebe webhooks e cria ligações a commits
     e PR")
}

Rel(spa, rest, "REST/JSON", "HTTPS")
Rel(rest, pedidos, "usa")
Rel(rest, prior, "usa")
Rel(rest, backlog, "usa")
Rel(rest, req, "usa")
Rel(rest, roadmap, "usa")
Rel(rest, anal, "usa")
Rel(rest, colab, "usa")
Rel(rest, contas, "autentica com")
Rel(pedidos, iaad, "pede triagem")
Rel(req, iaad, "pede esboço de User Story")
Rel(iaad, ia, "HTTPS/JSON")
Rel(pedidos, redis, "coloca tarefa de triagem")
Rel(colab, redis, "coloca tarefa de e-mail")
Rel(github, ghad, "Webhook HTTPS")
Rel(ghad, req, "liga commits a User Stories")
Rel_D(anal, db, "todos os módulos usam o ORM", "Django ORM")
@enduml
```